

AshaLink / ApkaSaathi

AshaLink is an elderly-care web platform built with Next.js and a Python ML backend. The project combines AI companionship, activity monitoring, medicine management, emergency workflows, neighborhood support, and caregiver reporting into a privacy-first dashboard experience.

8,399**APP AND BACKEND SOURCE LINES**

Across TypeScript, TSX, and Python files in `app/` and `backend/`.

12**NEXT.JS API ROUTES**

Voice, RAG, SOS, medicine AI, transcription, caregiver alerts, and ML proxy routes.

5,699**RAG KNOWLEDGE ENTRIES**

Stored in `public/rag_knowledge_base.json` and retrieved by the companion route.

What the project does

- Monitors electricity usage as a non-intrusive signal for daily routine changes, supported by an ESP32 hardware prototype.
- Provides Saathi, an AI companion with chat, voice, GoEmotions-based mood trend analysis, crisis detection, and report downloads.
- Supports medicine CRUD, AI medicine identification, local OCR, reminders, and medicine-specific RAG Q&A.
- Manages SOS contacts, Twilio calls/SMS, live location sharing, volunteers, and neighborhood help simulation.
- Aggregates caregiver insights from local mood, anomaly, medicine, and health-keyword data.

The dominant design choice is local-first persistence. Chat history, medicines, contacts, electricity history, reminder state, and caregiver settings are stored in browser `localStorage`, while cloud services are used only for selected AI, voice, and emergency actions.

Primary technology stack

Next.js 16 App Router

React 19

TypeScript

Tailwind CSS

Recharts

Leaflet

Framer Motion

Gemini

Groq

ElevenLabs

Twilio

FastAPI

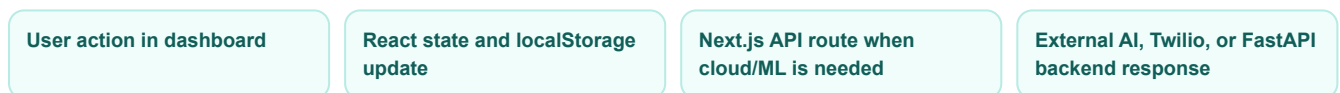
Application Structure and Data Flow

The app uses the Next.js App Router. The landing page is composed from marketing and onboarding components, while the dashboard routes host the operational product areas. API routes under `app/api` integrate with external AI and communication services. A separate FastAPI backend loads serialized ML models and exposes prediction endpoints consumed through the Next.js proxy route.

Top-level structure

AREA	PURPOSE	IMPORTANT FILES
Landing experience	Product introduction, login modal, feature sections, CTA.	<code>app/page.tsx</code> , <code>Hero.tsx</code> , <code>Features.tsx</code> , <code>CompetitiveEdge.tsx</code>
Dashboard shell	Shared dashboard navigation and route layout.	<code>app/dashboard/layout.tsx</code> , <code>DashboardNavbar.tsx</code>
Monitoring	Electricity charting, Z-score detection, adaptive hourly thresholds, ML toggle.	<code>app/dashboard/page.tsx</code> , <code>public/test_data.csv</code>
AI companion	Voice avatar, text RAG chat, GoEmotions-based mood trend analysis, reports, Smart SOS.	<code>app/dashboard/avatar/page.tsx</code> , <code>HumanSathi.tsx</code> , <code>RagChat.tsx</code> , <code>MoodChart.tsx</code>
Medicines	Medicine records, camera upload, cloud AI scan, local OCR, RAG answers, reminders.	<code>app/dashboard/medicines/page.tsx</code> , <code>medicine-identify</code> , <code>medicine-rag</code>
Safety and caregiver	SOS, neighbors, volunteers, location sharing, caregiver summary and downloads.	<code>community/page.tsx</code> , <code>caregiver/page.tsx</code> , <code>HealthAlerts.tsx</code>

Client-to-service flow



The frontend does most routine work locally: chart statistics, medicine state, health keyword scans, reports, emergency contact management, and community simulation. Server routes are thin adapters around third-party APIs or the local ML server. This keeps the UI fast for demos and makes privacy easy to explain.

API surface

- `/api/rag-chat` : loads the counseling knowledge base, retrieves similar conversations, applies crisis detection, and generates the Saathi response through Gemini.
- Mood trend analysis: GoEmotions-trained classifiers produce emotion labels and well-being scores used by the companion and caregiver charts.
- `/api/medicine-identify` and `/api/medicine-rag` : Groq Vision and Groq LLM medicine workflows.
- `/api/sos` , `/api/sos-location` , `/api/notify-caregiver` : Twilio voice/SMS emergency and caregiver notifications.
- `/api/transcribe` and `/api/tts` : Groq Whisper and ElevenLabs speech support.
- `/api/ml-anomaly` : proxy to the FastAPI anomaly prediction service on port 8000.

Dashboard Features and User Workflows

The dashboard is organized around practical elderly-care workflows rather than a single monolithic page. Each section has its own local state model and reportable outputs, which makes the project easy to demo module by module.

Activity monitoring

`app/dashboard/page.tsx` reads hourly electricity readings from `public/test_data.csv`, stores them into `electricity_history`, and computes per-hour adaptive ranges. If an hour has at least three readings, it uses mean and sample standard deviation; otherwise, it falls back to hardcoded ranges. The chart marks anomalies when `|Z-score| > 2`.

- The dashboard keeps only the last 30 days of hourly readings.
- The user can switch from statistical Z-score detection to ML ensemble mode.
- The UI explains peak usage, anomaly counts, date span, and whether adaptive or fallback thresholds are active.

Hardware prototype

The physical monitoring layer uses an `ESP32` development board on a breadboard with jumper wires, resistors, an OLED display module, a relay/driver module, a DC cooling fan, and a cardboard base. The Arduino sketch reads voltage from pin `34` through a divider and current from pin `35` using calibrated current-sensor values, then calculates power and accumulated energy.

- The OLED shows live voltage, current, power, and energy every second.
- The serial output prints the same readings for debugging and data capture.
- The relay/driver and fan create a visible load/control demonstration for the energy-monitoring concept.

AI companion

`app/dashboard/avatar/page.tsx` coordinates the Saathi companion. It persists messages in `avatar_messages`, attaches GoEmotions-derived mood signals to user messages, displays mood trends, and reacts to crisis metadata from the RAG route.

- `HumanSathi.tsx` provides the voice-oriented companion UI.
- `RagChat.tsx` sends text messages, recent history, and the elder name to the RAG API.
- `MoodChart.tsx` visualizes GoEmotions-derived well-being scores on a 1-10 scale.
- `SeedDemoData.tsx` can create realistic sample data for demos and caregiver reports.

Medicine management

`app/dashboard/medicines/page.tsx` is one of the largest feature files. It manages medicine CRUD, stores records in `user_medicines`, tracks reminders in `medicine_reminders`, supports image upload, and offers both cloud AI recognition and local OCR extraction.

- Cloud scanner sends base64 images to Groq Vision and expects structured JSON about medicine name, purpose, dosage, warnings, and disclaimer.
- Local scanner uses Tesseract.js WebAssembly in the browser and regex extraction for dosage/manufacture details.
- The RAG medicine Q&A sends only the user's saved medicine list as context, reducing hallucinated advice.

Safety and community

`app/dashboard/community/page.tsx` brings together emergency contacts, a Leaflet neighbor map, volunteer signup, help-request simulation, caregiver notification, siren audio, SOS calls/SMS, and location sharing. Distances are calculated with the Haversine formula.

Intelligence Layer

The codebase uses a hybrid intelligence strategy: browser-side rules for low-risk privacy-sensitive checks, hosted LLMs for language/vision/speech, and local Python models for structured anomaly and emotion prediction.

RAG companion route

`app/api/rag-chat/route.ts` loads `public/rag_knowledge_base.json` into memory, tokenizes the user's message, compares it against counseling contexts, retrieves the top five matching examples, and appends them to the system prompt. The final generation is performed with `gemini-2.5-flash`.

- The retrieval is TF-IDF-inspired keyword overlap rather than vector embeddings.
- The system prompt is elderly-care specific, India-aware, bilingual-friendly, and includes medical refusal rules.
- The route accepts recent chat history and a saved user name for personalization.

Smart SOS and mood trends

Crisis detection is paired with model-based mood trend analysis. The RAG route checks every companion message for critical self-harm, high-severity medical, and moderate emotional distress keywords. The GoEmotions model layer produces emotion labels and well-being scores so dangerous or declining patterns can be surfaced in the dashboard.

LAYER	METHOD	OUTPUT
RAG crisis detection	Keyword severity tiers	Structured crisis metadata returned to the UI
Mood trend model	GoEmotions-trained classifiers with elderly-care grouping	Emotion labels, well-being score, concern flags
Health alerts	Browser-only keyword category counts	Normal, attention, or urgent weekly alert category

FastAPI ML backend

`backend/main.py` starts a FastAPI service with CORS for localhost Next.js. It loads two predictor classes at startup: `AnomalyPredictor` and `EmotionPredictor`.

- `AnomalyPredictor` engineers nine features: kWh, cyclical hour/day encodings, weekend flag, rolling mean, rolling standard deviation, and kWh ratio.
- It runs Isolation Forest, Random Forest, and Gradient Boosting, then uses 2-of-3 majority voting.
- `EmotionPredictor` loads TF-IDF, a MultiLabelBinarizer, four one-vs-rest classifiers, and a macro sentiment classifier trained around GoEmotions labels.
- The emotion model returns primary emotion, detected emotions, top scores, macro sentiment, a well-being score, and elderly-care concern flags.

Saved model inventory

The repository contains serialized anomaly models, scalers, sequence autoencoders, and GoEmotions classifiers under `saved_models/` and `saved_models_goemotion/`. This makes the backend self-contained for local inference once Python dependencies are installed.

External Services, Storage, and Safety Posture

AshaLink depends on several cloud APIs, but the code is careful to keep many routine operations local. The practical privacy story is strongest around local OCR, localStorage persistence, local charting, and browser-side keyword scanning.

External integrations

SERVICE	USED FOR	ROUTES/COMPONENTS
Gemini	Companion RAG generation.	<code>rag-chat</code>
Groq	Medicine vision, medicine RAG, and Whisper transcription.	<code>medicine-identify</code> , <code>medicine-rag</code> , <code>transcribe</code>
ElevenLabs	Text-to-speech and voice cloning.	<code>tts</code> , <code>clone</code>
Twilio	SOS voice calls, SMS alerts, caregiver notifications, location links.	<code>sos</code> , <code>sos-location</code> , <code>notify-caregiver</code>
HeyGen	Streaming avatar session token.	<code>heygen</code>
Tesseract.js	In-browser OCR for medicine images.	<code>medicines/page.tsx</code>

Local data keys

- `avatar_messages` : companion transcript and GoEmotions-derived mood metadata.
- `electricity_history` : recent hourly electricity readings for adaptive thresholding.
- `user_medicines` and `medicine_reminders` : medicine records and reminder toggles.
- `sos_contacts` , `community_neighbours` , and `caregiver_settings` : safety contacts, local support network, and caregiver settings.
- `elder_name` and legacy `caregiver_phone` : personalization and older notification flow compatibility.

Security and maintainability observations

- Most secrets are read from environment variables, but `app/api/clone/route.ts` contains a hardcoded ElevenLabs API key comment. That should be rotated and moved to environment configuration before any public deployment.
- Several files include demo-ready default data, phone numbers, and client-only localStorage flows. This is suitable for prototype evaluation, but production would need authenticated user accounts and server-side access controls.
- The README mentions Groq for RAG, while the current companion route uses Gemini. The documentation should be updated so reviewers see the actual implementation.
- The project has two anomaly paths: browser Z-score detection and backend ML ensemble. This is a strength for demos, but the UI/report should clearly label which result source is active.

Best current positioning: a strong hackathon/prototype architecture for elderly-care assistance, with local-first privacy choices and credible AI/ML demonstrations. Main production gaps are authentication, secret hygiene, durable storage, audit trails, and clinical/legal safety review.

Strengths, Risks, and Next Steps

Project strengths

- Broad but coherent feature set around elderly independence, caregiver visibility, and emergency response.
- Clear separation between dashboard modules, API integrations, and Python model inference.
- Strong demo value: seeded data, charts, maps, voice, OCR, medicine AI, SOS, and downloadable reports.
- Good algorithm explainability: Z-score thresholds, Haversine distance, local keyword dictionaries, model majority voting, and RAG source display.
- Privacy-aware implementation choices, especially local OCR, GoEmotions mood modeling, health keyword scanning patterns, and browser persistence.

Important risks

- Hardcoded API credential in `app/api/clone/route.ts` must be treated as exposed.
- No authentication or user isolation is visible in the current app; browser storage is per-device, not per-account.
- Emergency and mental-health features need careful disclaimers, operational testing, and regional emergency-number handling before real-world use.
- Saved ML models are present, but training provenance, evaluation metrics, and dataset documentation are limited in the application surface.
- Large feature files such as medicines, community, dashboard, and avatar pages would benefit from splitting into smaller components and hooks.

Recommended next development steps

PRIORITY	WORK ITEM	WHY IT MATTERS
1	Rotate the hardcoded ElevenLabs key and remove it from source.	Prevents misuse and makes the repo safer to share.
2	Add authentication and move sensitive user data to a protected backend store.	Supports real caregiver/elder accounts and cross-device continuity.
3	Align README claims with code, especially Gemini versus Groq usage in companion RAG.	Improves evaluator trust and avoids implementation confusion.
4	Add tests for Z-score parsing, crisis detection, medicine OCR extraction, and API validation.	Protects the highest-risk logic from regressions.
5	Refactor large dashboard pages into hooks, panels, and service helpers.	Makes future development easier and reduces merge risk.

One-paragraph conclusion

AshaLink is a substantial elderly-care prototype with real product thinking behind it. The project demonstrates multiple assistive-care pillars: ESP32-based activity sensing, companion conversation, medicine support, emergency escalation, community assistance, and caregiver reporting. Its strongest technical story is the blend of physical energy monitoring, explainable local logic, cloud AI, and local ML inference. With secret cleanup, authentication, stronger tests, and some modular refactoring, it can move from a feature-rich demo toward a safer and more maintainable care platform.

BEST DEMO PATH

Seed data, show dashboard anomalies, chat with Saathi, scan medicine, trigger caregiver/SOS flow, then open caregiver report.

BEST TECHNICAL HIGHLIGHT

Show the ESP32 power monitor, then compare Z-score anomaly detection with the FastAPI ML ensemble.

BEST PRIVACY HIGHLIGHT

LocalStorage, local OCR, browser health keyword analysis, and no camera/wearable dependency for activity monitoring.